

	N=1	N=2	N=4	N=15
Symbolic prediction				
LTN	80.54 $\pm$ 23.33	77.54 $\pm$ 35.55	T/O	T/O
DeepProbLog	97.20 $\pm$ 0.50	95.20 $\pm$ 1.70	T/O	T/O
DPLA*	88.90 $\pm$ 14.80	83.60 $\pm$ 23.70	T/O	T/O
DeepStochLog	97.90 $\pm$ 0.10	96.40 $\pm$ 0.10	92.70 $\pm$ 0.60	T/O
Embed2Sym	97.62 $\pm$ 0.29	93.81 $\pm$ 1.37	91.65 $\pm$ 0.57	60.46 $\pm$ 20.36
A-NESI (predict)	97.66 $\pm$ 0.21	95.96 $\pm$ 0.38	92.56 $\pm$ 0.79	75.90 $\pm$ 2.21
A-NESI (explain)	97.37 $\pm$ 0.32	96.04 $\pm$ 0.46	92.11 $\pm$ 1.06	76.84 $\pm$ 2.82
A-NESI (pruning)	97.57 $\pm$ 0.27	95.82 $\pm$ 0.33	92.40 $\pm$ 0.68	76.46 $\pm$ 1.39
A-NESI (no prior)	76.54 $\pm$ 27.38	95.67 $\pm$ 0.53	44.58 $\pm$ 38.34	0.03 $\pm$ 0.09
Neural prediction				
Embed2Sym	97.34 $\pm$ 0.19	84.35 $\pm$ 6.16	0.81 $\pm$ 0.12	0.00
A-NESI (predict)	97.66 $\pm$ 0.21	95.95 $\pm$ 0.38	92.48 $\pm$ 0.76	54.66 $\pm$ 1.87
A-NESI (explain)	97.37 $\pm$ 0.32	96.05 $\pm$ 0.47	92.14 $\pm$ 1.05	61.77 $\pm$ 2.37
A-NESI (pruning)	97.57 $\pm$ 0.27	95.82 $\pm$ 0.33	92.38 $\pm$ 0.64	59.88 $\pm$ 2.95
A-NESI (no prior)	76.54 $\pm$ 27.01	95.28 $\pm$ 0.62	40.76 $\pm$ 34.29	0.00 $\pm$ 0.00
Reference	98.01	96.06	92.27	73.97

Table 1: Test accuracy of predicting the correct sum on the Multi-digit MNISTAdd task. “T/O” (timeout) represent computational timeouts. Reference accuracy approximates the accuracy of an MNIST predictor with 0.99 accuracy using 0.99^{2N} . Bold numbers signify the highest average accuracy for some N within the prediction categories. *predict* is the prediction-only variant, *explain* is the explainable variant, *pruning* adds symbolic pruning (see Appendix H), and *no prior* is the prediction-only variant *without* the prior $p(\mathbf{P})$.

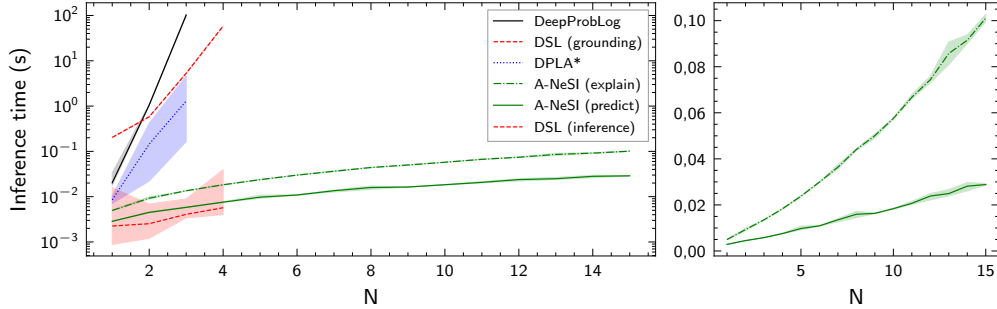


Figure 4: Inference time for a single input $\mathbf{x}$ for different nr. of digits. The left plot uses a log scale, and the right plot a linear scale. DSL stands for DeepStochLog. We use the GM variant of DPLA*.

neural prediction), it is much more accurate already for $N = 2$. Therefore, A-NESI’s training loop allows training a prediction network with high accuracy on large-scale problems.

Comparing the different A-NESI variants, we see the prediction-only, explainable and pruned variants perform quite similarly, with significant differences only appearing at $N = 15$ where the explainable and pruning variants outperform the predict-only model, especially on neural prediction. However, when removing the prior $p(\mathbf{P})$, the performance degrades quickly. The prediction model sees much fewer beliefs $\mathbf{P}$ than when sampling from a (high-entropy) prior $p(\mathbf{P})$. A second and more subtle reason is that at the beginning of training, all the beliefs $\mathbf{P} = f_{\theta}(\mathbf{x})$ will be uniform because the perception model is not yet trained. Then, the prediction model learns to ignore the input belief $\mathbf{P}$.

Figure 4 shows the runtime for inference on a single input $\mathbf{x}$. Inference in DeepProbLog (corresponding to exact inference) increases with a factor 100 as N increases, and DPLA* (another approximation) is not far behind. Inference in DeepStochLog, which uses different semantics, is efficient due to caching but requires a grounding step that is exponential both in time and memory. We could not ground beyond $N = 4$ because of memory issues. A-NESI avoids having to perform grounding altogether: it scales slightly slower than linear. Furthermore, it is much faster in practice as parallelizing the computation of multiple queries on GPUs is trivial.

	N=4	N=9
CNN	51.50 $\pm$ 3.34	51.20 $\pm$ 2.20
Exact inference	86.70 $\pm$ 0.50	T/O
NeuPSL	89.7 $\pm$ 2.20	51.50 $\pm$ 1.37
A-NESI (symbolic prediction)	89.70 $\pm$ 2.08	62.15 $\pm$ 2.08
A-NESI (neural prediction)	89.80 $\pm$ 2.11	62.25 $\pm$ 2.20

Table 2: Test accuracy of predicting whether a grid of numbers is a Sudoku. For A-NeSI, we used the prediction-only variant.

4.2 Visual Sudoku Puzzle Classification

In this task, the goal is to recognize whether an $N \times N$ grid of MNIST digits is a Sudoku. We have 100 examples of correct and incorrect grids from [4]. We use the same MNIST classifier as in the previous section. We treat $\mathbf{w}$ as a grid of digits in $\{1, \dots, N\}$ and designed an easy-to-learn representation of the label. For sudokus, all pairs of digits $\mathbf{w}_i, \mathbf{w}_j$ in a row, column, and block must differ. For each such pair, a dimension in $\mathbf{y}$ is 1 if different and 0 otherwise, and the symbolic function c computes these interactions from $\mathbf{w}$. The prediction model is a single MLP that takes the probabilities for the digits $\mathbf{P}_i$ and $\mathbf{P}_j$ and predicts the probability that those represent different digits. For additional details, see Appendix I.2.

Table 2 shows the accuracy of classifying Sudoku puzzles. A-NESI is the only method to perform better than random on 9×9 sudokus. Exact inference cannot scale to 9×9 sudokus, while we were able to run A-NESI for 3000 epochs in 38 minutes on a single NVIDIA RTX A4000.

4.3 Warcraft Visual Path Planning

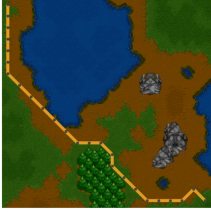


Figure 5: Example of the Warcraft task.

	12 $\times$ 12	30 $\times$ 30
ResNet18 [43]	23.0 $\pm$ 0.3	0.0 $\pm$ 0.0
SPL [1]	78.2	T/O
I-MLE [41]	97.2 $\pm$ 0.5	93.7 $\pm$ 0.6
RLOO [29]	43.75 $\pm$ 12.35	12.59 $\pm$ 16.38
A-NESI	94.57 $\pm$ 2.27	17.13 $\pm$ 16.32
A-NESI + RLOO	98.96 $\pm$ 1.33	67.57 $\pm$ 36.76

Table 3: Test accuracy of predicting the lowest-cost path on the Warcraft Visual Path Planning task. For A-NeSI, we used the prediction-only variant.

The Warcraft Visual Path Planning task [43] is to predict a shortest path from the top left corner of an $N \times N$ grid to the bottom right, given an image from the Warcraft game (Figure 5). The perception model has to learn the cost of each tile of the Warcraft image. We use a small CNN that takes the tile i, j (a $3 \times 8 \times 8$ image) and outputs a distribution over 5 possible costs. The symbolic function $c(\mathbf{w})$ is Dijkstra’s algorithm and returns a shortest path $\mathbf{y}$, which we encode as a sequence of the 8 (inter)-cardinal directions starting from the top-left corner. We use a ResNet18 [22] as the prediction model, which we train to predict the next direction given beliefs of the tile costs $\mathbf{P}$ and the current location. We pretrain the prediction model on a fixed prior and train the perception model with a frozen prediction model. See Appendix I.3 for additional details.

Table 3 presents the accuracy of predicting a shortest path. A-NESI is competitive on 12×12 grids but struggles on 30×30 grids. We believe this is because the prediction model is not accurate enough, resulting in gradients with too high bias. Still, A-NESI finds short paths, is far better than a pure neural network, and can scale to 30×30 grids, unlike SPL [1], which uses exact inference. Since both SPL and I-MLE [41] have significantly different setups (see Appendix I.3.2), we added experiments using REINFORCE with the leave-one-out baseline (RLOO, [29]) that we implemented with the Stochastic PyTorch library [53]. We find that RLOO has high variance in its performance. Since RLOO is unbiased with high variance and A-NESI is biased with no variance, we also tried running A-NESI and RLOO simultaneously. Interestingly, this is the best-performing method on the 12×12 grid, and has competitive performance on 30×30 grids, albeit with high variance (6 out 10 runs get to an accuracy between 93.3% and 98.5%, while the other runs are stuck around 25% accuracy).

5 Related work

A-NESI can approximate multiple PNL methods [16]. DeepProbLog [35] performs symbolic reasoning by representing w as ground facts in a Prolog program. It enumerates all possible proofs of a query y and weights each proof by $p(w|P)$. NeurASP [58] is a PNL framework closely related to DeepProbLog, but is based on Answer Set Programming [12]. Some methods consider constrained structured output prediction [19]. In Appendix B, we discuss extending A-NESI to this setting. Semantic Loss [57] improves learning with a loss function but does not guarantee that formulas are satisfied at test time. Like A-NESI, Semantic Probabilistic Layers [1] solves this with a layer that performs constrained prediction. These approaches perform exact inference using probabilistic circuits (PCs) [60]. Other methods perform approximate inference by only considering the top- k proofs in PCs [36, 23]. However, finding those proofs is hard, especially when beliefs have high entropy, and limiting to top- k significantly reduces performance. Other work considers MCMC approximations [31]. Using neural networks for approximate inference ensures computation time is constant and independent of the entropy of P or long MCMC chains.

Other neurosymbolic methods use fuzzy logics [5, 17, 15, 18], which are faster than PNL with exact inference. Although traversing ground formulas is linear time, the grounding is itself often exponential [37], so the scalability of fuzzy logics often fails to deliver. A-NESI is polynomial in the number of ground atoms and does not traverse the ground formula. Furthermore, background knowledge is often not fuzzy [54, 20], and fuzzy semantics does not preserve classical equivalence.

A-NESI performs gradient estimation of the WMC problem. We can extend our method to biased but zero-variance gradient estimation by learning a distribution over function outputs (see Appendix C). Many recent works consider continuous relaxations of discrete computation to make them differentiable [42, 24] but require many tricks to be computationally feasible. Other methods compute MAP states to compute the gradients [41, 11, 47] but are restricted to integer linear programs. The score function (or ‘REINFORCE’) gives unbiased yet high-variance gradient estimates [40, 53]. Variance reduction techniques, such as memory augmentation [32] and leave-one-out baselines [29], exist to reduce this variance.

6 Conclusion, Discussion and Limitations

We introduced A-NESI, a scalable approximate method for probabilistic neurosymbolic learning. We demonstrated that A-NESI scales to combinatorially challenging tasks without losing accuracy. A-NESI can be extended to include explanations and hard constraints without loss of performance.

However, there is no ‘free lunch’: when is A-NESI a good approximation? We discuss three aspects of learning tasks that could make it difficult to learn a strong and efficient inference model.

- **Dependencies of variables.** When variables in world w are highly dependent, finding an informative prior is hard. We suggest then using a prior that can incorporate dependencies such as a normalizing flow [45, 14] or deep generative models [51] over a Dirichlet distribution.
- **Structure in symbolic reasoning function.** We studied reasoning tasks with a relatively simple structure. Learning the inference model will be more difficult when the symbolic function c is less structured. Studying the relation between structure and learnability is interesting future work.
- **Problem size.** A-NESI did not perfectly train the prediction model in more challenging problems, which is evident from the divergence in performance between symbolic and neural prediction for 15 digits in Table 1. We expect its required parameter size and training time to increase with the problem size.

Promising future avenues are studying if the explanation model produces helpful explanations [50], extensions to continuous random variables [21] (see Appendix C for an example), and extensions to unnormalized distributions such as Markov Logic Networks [46], as well as (semi-) automated A-NESI solutions for neurosymbolic programming languages like DeepProbLog [35].